

Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение
высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

ФАКУЛЬТЕТ ПРИКЛАДНОЙ ИНФОРМАТИКИ

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ
по дисциплине «Мобильные и сетевые технологии»
НА ТЕМУ

Жадные алгоритмы. Динамическое программирование
15 Вариант

студент Северюхин Андрей Аркадьевич K3341
(Фамилия, Имя, Отчество) (группа)

Направление подготовки (специальность) 09.03.03
(код, наименование)

Мобильные и сетевые технологии

Квалификация Бакалавр
(бакалавр, магистр)*

Преподаватель Ромакина О.М.
(Фамилия, И. О.) (Подпись)

Дата выполнения: 21.09.2026

Санкт-Петербург
2026 г.

1 задача.

Максимальная стоимость добычи (0.5 балла)

Вор находит гораздо больше добычи, чем может поместиться в его сумке. Помогите ему найти самую ценную комбинацию предметов, предполагая, что любая часть предмета добычи может быть помещена в его сумку.

Цель - реализовать алгоритм для задачи о дробном рюкзаке.

Формат ввода / входного файла (input.txt). В первой строке входных данных задано целое число n - количество предметов, и W - вместимость сумки. Следующие n строк определяют значения веса и стоимости предметов. В i -ой строке содержатся целые числа p_i и w_i – стоимость и вес i -го предмета, соответственно.

Ограничения на входные данные.

$1 \leq n \leq 10^3$, $0 \leq W \leq 2 \cdot 10^6$, $0 \leq p_i \leq 2 \cdot 10^6$, $0 \leq w_i \leq 2 \cdot 10^6$ для всех $1 \leq i \leq n$.

Все числа - целые.

Формат вывода / выходного файла (output.txt). Выведите максимальное значение стоимости долей предметов, которые помещаются в сумку. Абсолютная погрешность между ответом вашей программы и оптимальным значением должно быть не более 10^{-3} . Для этого выведите свой ответ как минимум с четырьмя знаками после запятой (иначе ваш ответ, хотя и будет рассчитан правильно, может оказаться неверным из-за проблем с округлением).

Ограничение по времени. 2 сек.

Описание решения

Пусть есть оптимальное решение O и наше решение A .

Пусть у OPT товара с максимальной удельной стоимостью:

- $\min(p_i, W)$ - т.е. максимально возможное количество. Тогда этот шаг совпадает с нашим, и OPT уже согласуется с A .
- **НЕ** $\min(p_i, W)$ - т.е. товара с максимальной удельной стоимостью можно было взять ещё. Тогда в O есть какой-то другой товар (иначе сумка не полная и O можно было бы улучшить - противоречие). Заменяем часть этого товара на недостающее количество самого дорогого по удельной стоимости. Общий вес не изменится, а стоимость станет не хуже. Но лучше, чем O , быть не может - значит, стоимость осталась ровно такой же, и мы получили другое оптимальное решение, в котором самого дорогого товара взято максимально возможное количество.

Итого: в обоих случаях существует оптимальное решение, согласованное с нашим первым шагом. Это означает, что наш жадный шаг безопасен.

После этого шага остаётся задача того же вида: те же товары без взятого, вместимость $W - \min(w_i, W)$. Применяем к ней то же рассуждение - её первый шаг тоже безопасен. И так далее.

Асимптотика $O(n \log n)$

Решение

```
n, W = map(int, input().split())
arr = [tuple(map(int, input().split())) for _ in range(n)]
arr.sort(key=lambda x: x[0] / x[1], reverse=True)
result = 0

for item in arr:
    if W <= 0:
        break
    result += (item[0] / item[1]) * min(item[1], W)
    W -= item[1]

print(f"{result:.4f}")
```

Описание проведенных тестов

Ввиду отсутствия в задании ссылки на платформу для проверки, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).

```
dsa_labs — -zsh — 112x16
ok 01_sample1          0.01 c    7.7 M6
ok 02_sample2          0.01 c    7.7 M6
ok 03_zero_capacity     0.01 c    7.7 M6
ok 04_all_fits          0.01 c    7.7 M6
ok 05_zero_prices       0.01 c    7.7 M6
ok 07_repeating_fraction 0.01 c    7.8 M6
ok 08_equal_ratios      0.01 c    7.7 M6
ok 09_max_values        0.01 c    7.9 M6
ok 10_greedy_trap       0.01 c    7.8 M6
ok 12_tiny_weights      0.01 c    7.8 M6
ok big_11_random_max    0.01 c    8.0 M6
FAIL 1.1 Максимальная стоимость добычи lab1/1.1.py (1 из 12)
      06_zero_weight      ОШИБКА: ZeroDivisionError: division by zero

Всего запусков: 12, провалов: 1
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

В ходе выполнения данной задачи понял концепцию жадных алгоритмов а также научился доказывать корректность решения при помощи безопасного хода.

2 задача.

Заправки (0.5 балла)

Вы собираетесь поехать в другой город, расположенный в d км от вашего родного города. Ваш автомобиль может проехать не более m км на полном баке, и вы начинаете с полным баком. По пути есть заправочные станции на расстояниях $stop_1, stop_2, \dots, stop_n$ из вашего родного города. Какое минимальное количество заправок необходимо?

Формат ввода / входного файла (input.txt). В первой строке содержится d - целое число. Во второй строке - целое число m . В третьей строке находится количество заправок на пути - n . И, наконец, в последней строке - целые числа через пробел - остановки $stop_1, stop_2, \dots, stop_n$.

Ограничения на входные данные.

$1 \leq d \leq 10^5, 1 \leq m \leq 400, 1 \leq n \leq 300, 1 < stop_1 < stop_2 < \dots < stop_n < d$

Формат вывода / выходного файла (output.txt). Предполагая, что расстояние между городами составляет d км, автомобиль может проехать не более m км на полном баке, а заправки есть на расстояниях $stop_1, stop_2, \dots, stop_n$ по пути, выведите минимально необходимое количество заправок. Предположим, что машина начинает ехать с полным баком. Если до места назначения добраться невозможно, выведите -1 .

Ограничение по времени. 2 сек.

Описание решения

Тянуть до последней возможной заправки - безопасный ход.

Пусть OPT - оптимальный набор заправок, и его первая заправка стоит в точке x , а наша в точке y , причем y - самая дальняя достижимая, т.е. $x \leq y$. Заменяем x на y . Из y мы дотянем везде, куда дотягивали из x , потому что y дальше и бак после заправки все равно полный. Значит набор остался допустимым, а количество заправок не изменилось. Получили оптимальное решение, согласованное с нашим шагом.

Дальше остается такая же задача от точки y , и рассуждение повторяется. Значит наш ответ совпадает с оптимальным.

Асимптотика $O(n)$

Решение

```
d = int(input())
m = int(input())
n = int(input())
stops = list(map(int, input().split()))

stops = [0] + stops + [d]
refills = 0
pos = 0
i = 0

while pos < d:
    last = pos
    while i + 1 < len(stops) and stops[i + 1] - last <= m:
        i += 1
    if stops[i] == pos:
        refills = -1
        break
    pos = stops[i]
    if pos < d:
        refills += 1

print(refills)
```

Описание проведенных тестов

Ввиду отсутствия в задании ссылки на платформу для проверки, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).

```
dsa_labs — -zsh — 112x18
ok 01_sample1          0.01 c    7.8 M6
ok 02_sample2          0.01 c    7.8 M6
ok 03_sample3          0.01 c    7.7 M6
ok 04_exact_to_finish  0.01 c    7.7 M6
ok 05_exact_gaps       0.01 c    7.7 M6
ok 06_last_gap_too_long 0.01 c    7.7 M6
ok 07_first_unreachable 0.01 c    7.4 M6
ok 08_edge_plus_one    0.01 c    7.7 M6
ok 09_single_stop      0.01 c    7.8 M6
ok 10_dense_then_gap   0.01 c    7.7 M6
ok 11_max_stops        0.01 c    7.7 M6
ok 12_skip_near_stop   0.01 c    7.7 M6
ok 13_m_small          0.01 c    7.7 M6
ok 14_n_300_random     0.01 c    7.7 M6
OK 1.2 Заправки lab1/1.2.py 14 тестов, макс 0.01 с (лимит 2 с), 8 M6

Всего запусков: 14, провалов: 0
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

В ходе выполнения данной задачи закрепили умение доказывать безопасный ход.

12 задача.

Последовательность (1 балл)

Постановка задачи. Дана последовательность натуральных чисел $a_1, a_2, \dots, a_n$, и известно, что $a_i \leq i$ для любого $1 \leq i \leq n$. Требуется определить, можно ли разбить элементы последовательности на две части таким образом, что сумма элементов в каждой из частей будет равна половине суммы всех элементов последовательности.

Формат ввода / входного файла (input.txt). В первой строке входного файла находится одно целое число n . Во второй строке находится n целых чисел $a_1, a_2, \dots, a_n$.

Ограничения на входные данные.

$1 \leq n \leq 40000, 1 \leq a_i \leq i$.

Формат вывода / выходного файла (output.txt). В первую строку выходного файла выведите количество элементов последовательности в любой из получившихся двух частей, а во вторую строку через пробел номера этих элементов. Если построить такое разбиение невозможно, выведите -1.

Ограничение по времени. 2 сек.

Ограничение по памяти. 256 мб.

Описание решения

Если сумма нечетная, то деление нацело невозможно, сразу вернем -1. Иначе надо набрать $\text{target} = S / 2$.

Идем с конца и берем элемент, если он влезает в остаток target . Надо доказать, что так всегда можно добрать ровно до нуля.

Инвариант: target не больше суммы оставшихся элементов. В начале верно, т.к. $S / 2 \leq S$. Если элемент взяли, target и сумма оставшихся уменьшились на одно и то же. Если не взяли, значит $\text{target} < a_i$, а по условию $a_i \leq i$, значит $\text{target} \leq i - 1$. Но слева лежит $i - 1$ число, т.е. в сумме тоже не меньше $i - 1$. Инвариант сохраняется.

В конце $\text{target} \leq 0$, а отрицательным он быть не может, мы не берем элементы больше остатка, значит ровно 0.

Т.е. условие $a_i \leq i$ не дает элементу быть слишком большим по сравнению с тем, что левее, поэтому при четной сумме ответ есть всегда.

Асимптотика $O(n)$

Решение

```
n = int(input())
a = list(map(int, input().split()))
s = sum(a)

if s % 2 == 1:
    print(-1)
else:
    target = s // 2
    res = []
    for i in range(n - 1, -1, -1):
        if a[i] <= target:
            target -= a[i]
            res.append(i + 1)
    res.reverse()
    print(len(res))
    print(*res)
```

Описание проведенных тестов

Ввиду отсутствия в задании ссылки на платформу для проверки, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).

```
dsa_labs — -zsh — 112x16
ok 01_sample          0.01 c    7.6 Mб
ok 02_odd_sum         0.01 c    7.8 Mб
ok 03_n1              0.01 c    7.7 Mб
ok 04_two_ones        0.01 c    7.7 Mб
ok 05_two_odd         0.01 c    7.7 Mб
ok 06_all_ones_even   0.01 c    7.7 Mб
ok 07_all_ones_odd    0.01 c    7.7 Mб
ok 08_a_equals_i      0.01 c    7.8 Mб
ok 11_one_big_element 0.01 c    7.8 Mб
ok 12_all_ones_40000  0.02 c    9.4 Mб
ok big_09_max_random  0.02 c   12.3 Mб
ok big_10_max_a_equals_i 0.02 c   12.4 Mб
OK 1.12 Последовательность lab1/1.12.py 12 тестов, макс 0.02 с (лимит 2 с), 12 Мб

Всего запусков: 12, провалов: 0
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

В ходе данной работы научились работать с инвариантами.

16 задача.

Продавец (2 балла)

Постановка задачи. Продавец техники хочет объехать n городов, посетив каждый из них ровно один раз. Помогите ему найти кратчайший путь.

Формат ввода / входного файла (input.txt). Первая строка входного файла содержит натуральное число n – количество городов. Следующие n строк содержат по n чисел – длины путей между городами. В i -й строке j -е число – $a_{i,j}$ – это расстояние между городами i и j .

Ограничения на входные данные.

$1 \leq n \leq 13$, $0 \leq a_{i,j} \leq 10^6$, $a_{i,j} = a_{j,i}$, $a_{i,i} = 0$.

Формат вывода / выходного файла (output.txt). В первой строке выходного файла выведите длину кратчайшего пути. Во второй строке выведите n чисел – порядок, в котором нужно посетить города.

Ограничение по времени. 1 сек.

Ограничение по памяти. 256 мб.

Описание решения

Это задача коммивояжера, при $n = 13$ (самое большое число в рамках ограничений этой задачи) всего вариантов будет $13!$, что примерно 6 млрд. Поэтому решить перебором не получится (ну и также тема данной лабораторной - жадные алгоритмы и динамическое программирование). Применим последнее в данной задаче.

Если мы уже обошли какие-то города и стоим в городе v , то для продолжения пути не имеет значения, в каком порядке мы их обходили. Важно какие города посещены и где мы сейчас. Значит все перестановки с одинаковым набором и одинаковым концом можно собрать в одно состояние.

Сделаем обычный перебор с массивом `visited`, но результат запоминаем по ключу (набор посещенных, текущий город). Если такое состояние уже считали, второй раз не считаем.

Начинать можно из любого города, поэтому запускаем перебор n раз и берем минимум.

Маршрут восстанавливается через `choice` - для каждого состояния запоминаем, какой город оказался лучшим следующим, и идем по этим подсказкам от старта, по дороге отмечая города посещенными.

Асимптотика $O(2^n * n^2)$

Решение

```
n = int(input())
a = [list(map(int, input().split())) for _ in range(n)]

inf = float('inf')
visited = [False] * n
memo = {}
choice = {}
#пришлось пожертвовать оформлением кода тк код не влезал на страницу
def solve(v, cnt):
    if cnt == n: return 0
    key = (tuple(visited), v)
    if key in memo: return memo[key]
    best = inf
    best_u = -1
    for u in range(n):
        if not visited[u]:
            visited[u] = True
            cost = a[v][u] + solve(u, cnt + 1)
            visited[u] = False
            if cost < best:
                best = cost
                best_u = u
    memo[key] = best
    choice[key] = best_u
    return best

answer = inf
start = 0
for v in range(n):
    visited[v] = True
    cost = solve(v, 1)
    visited[v] = False
    if cost < answer:
        answer = cost
        start = v

path = [start + 1]
visited[start] = True
v = start
for _ in range(n - 1):
    u = choice[(tuple(visited), v)]
    visited[u] = True
    path.append(u + 1)
    v = u
print(answer)
print(*path)
```

Описание проведенных тестов

Ввиду отсутствия в задании ссылки на платформу для проверки, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).

```
dsa_labs — -zsh — 112x14
ok 01_sample          0.01 c    7.9 Mб
ok 02_n1              0.01 c    7.9 Mб
ok 03_n2              0.01 c    7.8 Mб
ok 04_all_zeros       0.01 c    7.8 Mб
ok 05_nearest_neighbour_trap 0.01 c    7.8 Mб
ok 06_n8_random       0.01 c    8.1 Mб
ok 07_n10_random      0.02 c    9.5 Mб
ok big_08_n13_max_weights 0.13 c   29.7 Mб
ok big_09_n13_random  0.13 c   29.8 Mб
ok big_10_two_clusters 0.06 c   18.1 Mб
OK 1.16 Продавец (TSP) lab1/1.16.py 10 тестов, макс 0.13 с (лимит 1 с), 30 Mб

Всего запусков: 10, провалов: 0
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

В ходе решения данной задачи познакомились с задачей коммивояжера. В рамках этой лабораторной работы эта задача - первая, в которой использовалось динамическое программирование, а не жадный алгоритм.

21 задача.

Игра в дурака (3 балла)

Постановка задачи. Петя очень любит программировать. Недавно он решил реализовать популярную карточную игру «Дурак». Но у Пети пока маловато опыта, ему срочно нужна Ваша помощь.

Как известно, в «Дурака» играют колодой из 36 карт. В Петиной программе каждая карта представляется в виде строки из двух символов, где первый символ означает ранг ('6', '7', '8', '9', 'T', 'J', 'Q', 'K', 'A') карты, а второй символ означает масть ('S', 'C', 'D', 'H'). Ранги перечислены в порядке возрастания старшинства.

Пете необходимо решить следующую задачу: сможет ли игрок, обладая набором из N карт, отбить M карт, которыми под него сделан ход? Для того чтобы отбиться, игроку нужно покрыть каждую из карт, которыми под него сделан ход, картой из своей колоды. Карту можно покрыть либо старшей картой той же масти, либо картой козырной масти. Если кроющаяся карта сама является козырной, то её можно покрыть только старшим козырем. Одной картой можно покрыть только одну карту.

Формат входного файла (input.txt). В первой строке входного файла находятся два натуральных числа N и M , а также символ R , означающий козырную масть. Во второй строке перечислены N карт, находящихся на руках у игрока. В третьей строке перечислены M карт, которые необходимо отбить. Все карты отделены друг от друга одним пробелом.

Ограничения на входные данные.

$N \leq 35$, $M \leq 4$, $M \leq N$.

Формат выходного файла (output.txt). В выходной файл выведите «YES» в случае, если отбиться можно, либо «NO», если нельзя.

Ограничение по времени. 1 сек.

Ограничение по памяти. 16 мб.

Описание решения (полный перебор)

Первым делом попробуем решить задачу полным перебором.
Асимптотика $O(N^M)$

Решение (полный перебор)

Решение полным перебором представлено ниже. Оно проходит по всем ограничениям и принимается на <https://acmp.ru/>. (см. описание проведенных тестов)

```
RANKS = '6789TJQKA'

data = input().split()
n, m, trump = int(data[0]), int(data[1]), data[2]
hand = input().split()
attack = input().split()

used = [False] * n

def beats(c, x):
    if x[1] == trump:
        return c[1] == trump and RANKS.index(c[0]) >
RANKS.index(x[0])
    if c[1] == x[1]:
        return RANKS.index(c[0]) > RANKS.index(x[0])
    return c[1] == trump

def solve(i):
    if i == m:
        return True
    for j in range(n):
        if not used[j] and beats(hand[j], attack[i]):
            used[j] = True
            if solve(i + 1):
                return True
            used[j] = False
    return False

if solve(0):
    print("YES")
else:
    print("NO")
```

Решение (динамическое программирование)

Однако поскольку тема этой лабораторной работы связана с динамическим программированием, то имеет смысл решить данную задачу именно этим способом.

Жадным алгоритмом решить не получится тк карту часто можно побить и своей мастью, и козырем. Если потратить лишний козырь, то на другую карту не хватит.

Карт соперника максимум 4. Поэтому будем следить, какие из них уже закрыты.

Набор закрытых карт будем хранить как кортеж из m флагов и держим множество всех наборов, которые достижимы.

Дальше берем карты руки по очереди. Каждую можно не использовать, тогда старые наборы остаются, или побить ей одну еще не закрытую карту, тогда появляется новый набор с поднятым флагом.

В конце смотрим, есть ли среди наборов тот, где закрыто все.

В общем перебор делал лишние операции. Допустим, первую карту мы побили валетом, а вторую дамой - или наоборот, первую дамой, вторую валетом. Перебор считает это двумя разными ветками и каждую разворачивает до конца. Хотя результат один и тот же - обе карты закрыты, валет и дама потрачены, дальше задача одинаковая.

Асимптотика $O(N * 2^M * M)$

Решение (динамическое программирование)

```
RANKS = '6789TJQKA'

data = input().split()
n, m, trump = int(data[0]), int(data[1]), data[2]
hand = input().split()
attack = input().split()

def beats(c, x):
    if x[1] == trump:
        return c[1] == trump and RANKS.index(c[0]) >
RANKS.index(x[0])
    if c[1] == x[1]:
        return RANKS.index(c[0]) > RANKS.index(x[0])
    return c[1] == trump

start = tuple([False] * m)
goal = tuple([True] * m)
states = {start}

for card in hand:
    new_states = set(states)
    for st in states:
        for i in range(m):
            if not st[i] and beats(card, attack[i]):
                nxt = list(st)
                nxt[i] = True
                new_states.add(tuple(nxt))
    states = new_states

if goal in states:
    print("YES")
else:
    print("NO")
```


Описание проведенных тестов

№1 - №20								Вперед »
ID	Дата	Автор	Задача	Язык	Результат	Тест	Время	Память
25474786	21.09.2026 15:10:05	Аминов Бехруз	0002	C++	Accepted		0,03	326 Кб
25474785	21.09.2026 15:10:04	Неизвестный	1086	Python	Accepted		0,046	344 Кб
25474784	21.09.2026 15:09:58	Неизвестный	0033	Python	Wrong answer	1		316 Кб
25474783	21.09.2026 15:09:57	Северюхин Андрей Аркадьевич	0698	Python	Accepted		0,046	316 Кб
25474782	21.09.2026 15:09:53	Умирбек Нрмейір	1489	C++	Compilation error			
25474781	21.09.2026 15:09:30	Аминов Бехруз	0002	C++	Presentation error	1		326 Кб
25474780	21.09.2026 15:09:15	Хасянжин Игорь	0970	Python	Wrong answer	1		304 Кб
25474779	21.09.2026 15:09:14	Пулатов Арсен	0754	C++	Accepted		0,03	322 Кб
25474778	21.09.2026 15:08:54	Неизвестный	0108	C++	Accepted		0,015	322 Кб
25474777	21.09.2026 15:08:53	Бақдаулет	2325	C++	Wrong answer	2	0,015	314 Кб
25474776	21.09.2026 15:08:51	Умирбек Нрмейір	1489	C++	Compilation error			
25474775	21.09.2026 15:08:51	Полтавский Александр Алексеевич	0004	Python	Accepted		0,046	348 Кб
25474774	21.09.2026 15:08:50	Неизвестный	1416	Python	Accepted		0,046	320 Кб
25474773	21.09.2026 15:08:43	Борисов Сергей Афанасьевич	0004	C++	Accepted		0,03	326 Кб
25474772	21.09.2026 15:08:43	Неизвестный	0773	C++	Accepted		0,015	334 Кб
25474771	21.09.2026 15:08:37	Мейрхан	1149	C++	Wrong answer	1		314 Кб
25474770	21.09.2026 15:08:35	Мендыбаев Асмир Айдосович	1123	C++	Accepted		0,03	326 Кб
25474769	21.09.2026 15:08:34	Хасянжин Игорь	0970	Python	Wrong answer	1		336 Кб
25474768	21.09.2026 15:08:33	Калкабек мансур	2005	C++	Wrong answer	2	0,015	318 Кб
25474767	21.09.2026 15:08:19	Мейрамбек Даулет	0516	C++	Wrong answer	3	0,015	326 Кб

№1 - №20								Вперед »
ID	Дата	Автор	Задача	Язык	Результат	Тест	Время	Память
25474798	21.09.2026 15:10:59	Аслан	0539	C++	Wrong answer	1		326 Кб
25474797	21.09.2026 15:10:56	Умирбек Нрмейір	1489	C++	Compilation error			
25474796	21.09.2026 15:10:52	Алим Бактияр	0108	C++	Accepted		0,015	326 Кб
25474795	21.09.2026 15:10:52	Умирбек Нрмейір	1489	C++	Compilation error			
25474794	21.09.2026 15:10:52	Северюхин Андрей Аркадьевич	0698	Python	Accepted		0,046	320 Кб
25474793	21.09.2026 15:10:47	Ключников Никита Алексеевич	0076	C++	Wrong answer	7	0,03	354 Кб
25474792	21.09.2026 15:10:39	ОСКЕНБАЙ	0754	C++	Wrong answer	3	0,015	334 Кб
25474791	21.09.2026 15:10:38	Борисов Сергей Афанасьевич	0755	C++	Wrong answer	1		330 Кб
25474790	21.09.2026 15:10:25	Орманов Жанторе Алтаевич	0511	C++	Accepted		0,03	322 Кб
25474789	21.09.2026 15:10:24	Попов	0011	C++	Wrong answer	1		318 Кб
25474788	21.09.2026 15:10:18	АБДИМАНАП КОЖАХМЕТ	1576	C++	Accepted		0,03	326 Кб
25474787	21.09.2026 15:10:14	Нурилла Джаксимуратов	0061	C++	Wrong answer	3	0,03	318 Кб
25474786	21.09.2026 15:10:05	Аминов Бехруз	0002	C++	Accepted		0,03	326 Кб
25474785	21.09.2026 15:10:04	Неизвестный	1086	Python	Accepted		0,046	344 Кб
25474784	21.09.2026 15:09:58	Неизвестный	0033	Python	Wrong answer	1		316 Кб
25474783	21.09.2026 15:09:57	Северюхин Андрей Аркадьевич	0698	Python	Accepted		0,046	316 Кб
25474782	21.09.2026 15:09:53	Умирбек Нрмейір	1489	C++	Compilation error			
25474781	21.09.2026 15:09:30	Аминов Бехруз	0002	C++	Presentation error	1		326 Кб
25474780	21.09.2026 15:09:15	Хасянжин Игорь	0970	Python	Wrong answer	1		304 Кб
25474779	21.09.2026 15:09:14	Пулатов Арсен	0754	C++	Accepted		0,03	322 Кб

Выводы

В ходе решения данной задачи удалось решить ее как полным перебором, так и оптимизировать решение при помощи динамического программирования.

Дополнительные задачи

3 задача.

Максимальный доход от рекламы (0.5 балла)

У вас есть n объявлений для размещения на популярной интернет-странице. Для каждого объявления вы знаете, сколько рекламодатель готов платить за один клик по этому объявлению. Вы настроили n слотов на своей странице и оценили ожидаемое количество кликов в день для каждого слота. Теперь ваша цель - распределить рекламу по слотам, чтобы максимизировать общий доход.

Постановка задачи. Даны две последовательности $a_1, a_2, \dots, a_n$ (a_i - прибыль за клик по i -му объявлению) и $b_1, b_2, \dots, b_n$ (b_i - среднее количество кликов в день i -го слота), нужно разбить их на n пар (a_i, b_j) так, чтобы сумма их произведений была максимальной.

Формат ввода / входного файла (input.txt). В первой строке содержится целое число n , во второй - последовательность целых чисел $a_1, a_2, \dots, a_n$, в третьей - последовательность целых чисел $b_1, b_2, \dots, b_n$.

Ограничения на входные данные.

$1 \leq n \leq 10^3$, $-10^5 \leq a_i, b_i \leq 10^5$, для всех $1 \leq i \leq n$.

Формат вывода / выходного файла (output.txt). Выведите максимальное значение $\sum_{i=1}^n a_i c_i$, где $c_1, c_2, \dots, c_n$ является перестановкой $b_1, b_2, \dots, b_n$.

Ограничение по времени. 2 сек.

Описание решения

Идея очевидная - самое дорогое объявление ставим в самый большой по кликам слот. То есть сортируем оба списка и перемножаем по порядку.

Докажем, что так можно. Пусть в оптимуме нашлась пара, где порядок нарушен: $a_1 < a_2$, но a_1 стоит в большем слоте, т.е. $b_1 > b_2$. Поменяем их местами. Было $a_1 * b_1 + a_2 * b_2$, стало $a_1 * b_2 + a_2 * b_1$. Разница равна $(a_2 - a_1) * (b_1 - b_2)$, оба множителя положительные, значит стало не хуже.

Получается, любую перевернутую пару можно развернуть, не ухудшив ответ. Значит существует оптимум, где вообще нет перевернутых пар, а это и есть две отсортированные последовательности.

Асимптотика $O(n \log n)$

Решение

```
n = int(input())
a = list(map(int, input().split()))
b = list(map(int, input().split()))

a.sort()
b.sort()

print(sum(a[i] * b[i] for i in range(n)))
```

Описание проведенных тестов

Ввиду отсутствия в задании ссылки на платформу для проверки, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).

```
dsa_labs — -zsh — 112x13
ok 01_sample1          0.01 c    7.8 Mб
ok 02_sample2          0.01 c    7.7 Mб
ok 03_all_negative     0.01 c    7.7 Mб
ok 04_zeros            0.01 c    7.7 Mб
ok 05_single_negative  0.01 c    7.7 Mб
ok 06_mixed_signs      0.01 c    7.7 Mб
ok 07_max_magnitude    0.01 c    7.8 Mб
ok 08_random_max       0.01 c    7.8 Mб
ok 09_sign_pairing     0.01 c    7.7 Mб
OK 1.3 Максимальный доход от рекламы lab1/1.3.py 9 тестов, макс 0.01 с (лимит 2 с), 8 Мб

Всего запусков: 9, провалов: 0
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

В ходе решения данной задачи укрепили, навыки доказательства корректности решения, и уяснили, что за долгими рассуждениями может стоять очень короткий код.

8 задача.

Расписание лекций (1 балл)

Постановка задачи. Вы наверно знаете, что в ИТМО лекции читают одни из лучших преподаватели мира. К сожалению, лекционных аудиторий у нас не так уж и много, особенно на Биржевой, поэтому каждый преподаватель составил список лекций, которые он хочет прочитать студентам. Чтобы студенты, в начале февраля, увидели расписание лекций, необходимо его составить прямо сейчас. И без вас нам здесь не справиться. У нас есть список заявок от преподавателей на лекции для одной из аудиторий. Каждая заявка представлена в виде временного интервала $[s_i, f_i)$ - время начала и конца лекции. Лекция считается открытым интервалом, то есть какая-то лекция может начаться в момент окончания другой, без перерыва. Необходимо выбрать из этих заявок такое подмножество, чтобы суммарно выполнить максимальное количество заявок. Учтите, что одновременно в лекционной аудитории, конечно же, может читаться лишь одна лекция.

Формат ввода / входного файла (input.txt). В первой строке вводится натуральное число N - общее количество заявок на лекции. Затем вводится N строк с описаниями заявок - по два числа в каждом s_i и f_i для каждой лекции i . Гарантируется, что $s_i < f_i$. Время начала и окончания лекции - натуральные числа, не превышают 1440 (в минутах с начала суток).

Ограничения на входные данные.

$$1 \leq N \leq 1000, 1 \leq s_i, f_i \leq 1440$$

Формат вывода / выходного файла (output.txt). Выведите одно число – максимальное количество заявок на проведение лекций, которые можно выполнить.

Ограничение по времени. 2 сек.

Ограничение по памяти. 256 мб.

Описание решения

Берем лекцию, которая заканчивается раньше всех. Она освобождает аудиторию раньше всех, а значит оставляет больше места остальным.

Проверим. Пусть есть оптимальное расписание, его первая лекция кончается в f_1 . Наша кончается не позже, мы же выбрали самую раннюю по окончанию. Заменяем ее нашей - все остальные лекции начинались после f_1 , а наша кончилась еще раньше, значит ничего не пересеклось, и количество лекций то же. Значит выбор ничего не испортил.

Дальше выкидываем все, что пересекается с взятой, и получаем такую же задачу. Рассуждение повторяется.

Асимптотика $O(N \log N)$

Решение

```
n = int(input())
lectures = [tuple(map(int, input().split())) for _ in range(n)]

lectures.sort(key=lambda x: x[1])

count = 0
last = 0

for s, f in lectures:
    if s >= last:
        count += 1
        last = f

print(count)
```

Описание проведенных тестов

Ввиду отсутствия в задании ссылки на платформу для проверки, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).

```
dsa_labs — -zsh — 112x14
ok 01_sample1          0.01 c    7.6 M6
ok 02_sample2          0.01 c    7.8 M6
ok 03_touching         0.01 c    7.7 M6
ok 04_identical        0.01 c    7.7 M6
ok 05_nested           0.01 c    7.7 M6
ok 06_long_plus_short  0.01 c    7.8 M6
ok 07_all_overlap      0.01 c    7.8 M6
ok 08_random_max       0.01 c    7.8 M6
ok 09_unit_chain       0.01 c    7.8 M6
ok 10_greedy_by_start_trap 0.01 c    7.7 M6
OK 1.8  Расписание лекций lab1/1.8.py 10 тестов, макс 0.01 с (лимит 2 с), 8 М6

Всего запусков: 10, провалов: 0
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

В ходе решения данной работы познакомились с классической задачей расписания, а также в очередной раз закрепили навык доказательства решения.

10 задача.

Яблоки (1 балл)

Постановка задачи. Алисе в стране чудес попались n волшебных яблок. Про каждое яблоко известно, что после того, как его съешь, твой рост сначала уменьшится на a_i сантиметров, а потом увеличится на b_i сантиметров. Алиса очень голодная и хочет съесть все n яблок, но боится, что в какой-то момент ее рост s станет равным нулю или еще меньше, и она пропадет совсем. Помогите ей узнать, можно ли съесть яблоки в таком порядке, чтобы в любой момент времени рост Алисы был больше нуля.

Формат ввода / входного файла (input.txt). В первой строке вводятся натуральные числа n и s – число яблок и начальный рост Алисы. В следующих n строках вводятся пары натуральных чисел a_i, b_i через пробел.

Ограничения на входные данные.

$1 \leq n \leq 1000, 1 \leq s \leq 1000, 1 \leq a_i, b_i \leq 1000.$

Формат вывода / выходного файла (output.txt). Если яблоки съесть нельзя, выведите число -1. Иначе выведите n чисел – номера яблок, в том порядке, в котором их нужно есть.

Ограничение по времени. 2 сек.

Ограничение по памяти. 256 мб.

Описание решения

Сколько бы яблок Алиса ни съела и в каком бы порядке, в конце рост будет один и тот же - все a вычтутся, все b прибавятся. От порядка зависит только то, не упадет ли рост до нуля посередине.

Риск возникает в момент между вычитанием a и прибавлением b , и чем больше a , тем он выше. Значит яблоки с большим a едим первыми, пока рост еще большой.

Проверим, что безопасно. Пусть есть правильный порядок, в котором где-то рядом стоят два яблока, и то, у которого a больше, идет вторым. Поменяем их местами. Рост до пары и после пары не изменится, сумма та же. А внутри пары станет только безопаснее: раньше мы сначала вычитали маленькое a , потом большое, теперь наоборот - и в промежуточной точке рост окажется не меньше, чем был. Значит порядок остался правильным.

Асимптотика $O(n \log n)$

Решение

```
n, s = map(int, input().split())
apples = [tuple(map(int, input().split())) for _ in range(n)]

order = sorted(range(n), key=lambda i: -apples[i][0])

for i in order:
    a, b = apples[i]
    if s - a <= 0:
        print(-1)
        break
    s = s - a + b
else:
    print(*[i + 1 for i in order])
```

Описание проведенных тестов

Ввиду отсутствия в задании ссылки на платформу для проверки, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).

```
dsa_labs — -zsh — 112x19

FAIL 01_sample1      0.01 c    7.6 Mб выведено -1, хотя порядок существует, например 1 3 2
ok    02_sample2      0.01 c    7.4 Mб
FAIL 03_order_by_a_trap 0.01 c    7.7 Mб выведено -1, хотя порядок существует, например 2 1
FAIL 04_all_gainers  0.01 c    7.7 Mб выведено -1, хотя порядок существует, например 1 2 3
ok    05_all_losers   0.01 c    7.7 Mб
ok    06_exact_zero   0.01 c    7.7 Mб
ok    07_single_ok    0.01 c    7.7 Mб
ok    08_single_bad   0.01 c    7.7 Mб
ok    09_max_tight    0.01 c    8.1 Mб
ok    10_max_impossible 0.01 c    7.9 Mб
ok    11_max_mixed    0.01 c    7.9 Mб
ok    12_one_valid_start 0.01 c    7.7 Mб
FAIL 1.10 Яблоки      lab1/1.10.py (3 из 12)
      01_sample1      выведено -1, хотя порядок существует, например 1 3 2
      03_order_by_a_trap выведено -1, хотя порядок существует, например 2 1
      04_all_gainers    выведено -1, хотя порядок существует, например 1 2 3

Всего запусков: 12, провалов: 3
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

11 задача.

Максимальное количество золота (1 балл)

Вам дается набор золотых слитков, и ваша цель - набрать как можно больше золота в свою сумку. Существует только одна копия каждого слитка, и для каждого слитка вы можете либо взять его, либо нет (т.е. вы не можете взять часть слитка).

Постановка задачи. Даны n золотых слитков, найдите максимальный вес золота, который поместится в сумку вместимостью W .

Формат ввода / входного файла (input.txt). Первая строка входных данных содержит вместимость W сумки и количество n золотых слитков. В следующей строке записано n целых чисел $w_0, w_1, \dots, w_{n-1}$, определяющие вес золотых слитков.

Ограничения на входные данные.

$$1 \leq W \leq 10^4, 1 \leq n \leq 300, 0 \leq w_0, \dots, w_{n-1} \leq 10^5$$

Формат вывода / выходного файла (output.txt). Выведите максимальный вес золота, который поместится в сумку вместимости W .

Ограничение по времени. 5 сек.

Обратите внимание, что в этой задаче все предметы имеют одинаковую стоимость на единицу веса по простой причине: все они сделаны из золота.

Описание решения

В отличие от первой задачи, дробную часть брать нельзя. Жадный алгоритм здесь работать не будет.

Перебрать все наборы нельзя тк их 2^{300} . Однако можно использовать следующее - нам неважно, какие именно слитки лежат в сумке, а важно только сколько они весят. Два разных набора весом 7 дальше ведут себя одинаково.

Заведем массив dp , где $dp[j]$ - сколько максимум золота помещается, если места ровно j . Берем слитки по очереди. Каждый либо не кладем, тогда все остается как было, либо кладем, и тогда к тому, что набиралось на месте $j - x$, добавляется сам слиток.

Места перебираем с конца, иначе один слиток положится в сумку несколько раз: мы сначала обновим маленькое место, а потом на большом воспользуемся этим же обновлением.

Асимптотика $O(n * W)$

Решение

```
W, n = map(int, input().split())
w = list(map(int, input().split()))

dp = [0] * (W + 1)

for x in w:
    for j in range(W, x - 1, -1):
        if dp[j - x] + x > dp[j]:
            dp[j] = dp[j - x] + x

print(dp[W])
```

Описание проведенных тестов

Ввиду отсутствия в задании ссылки на платформу для проверки, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).

```
dsa_labs — -zsh — 112x14
ok 01_sample 0.01 c 7.7 M6
ok 02_all_too_heavy 0.01 c 7.7 M6
ok 03_exact_fill 0.01 c 8.0 M6
ok 04_zero_weights 0.01 c 8.1 M6
ok 05_single_item 0.01 c 7.4 M6
ok 06_greedy_trap 0.01 c 7.7 M6
ok 07_parity_gap 0.01 c 8.0 M6
ok big_08_worst_time 0.37 c 8.2 M6
ok big_09_random_max 0.02 c 8.0 M6
ok big_10_dense_max 0.32 c 8.1 M6
OK 1.11 Максимальное количество золота lab1/1.11.py 10 тестов, макс 0.37 с (лимит 5 с), 8 M6

Всего запусков: 10, провалов: 0
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

В ходе данной работы еще раз воспользовались динамическим программированием, укрепив навык работы с этим способом решения.

17 задача.

Ход конем (2.5 балла)

Постановка задачи. Шахматная ассоциация решила оснастить всех своих сотрудников такими телефонными номерами, которые бы набирались на кнопочном телефоне ходом коня. Например, ходом коня набирается телефон 340-49-27. При этом телефонный номер не может начинаться ни с цифры 0, ни с цифры 8.

1	2	3
4	5	6
7	8	9
.	0	.

Напишите программу, определяющую количество телефонных номеров длины N , набираемых ходом коня. Поскольку таких номеров может быть очень много, выведите ответ по модулю 10^9 .

Формат ввода / входного файла (input.txt). Во входном файле записано одно целое число N .

Ограничения на входные данные.

$1 \leq N \leq 1000$.

Формат вывода / выходного файла (output.txt). Выведите в выходной файл искомое количество телефонных номеров по модулю 10^9 .

Ограничение по времени. 1 сек.

Ограничение по памяти. 256 мб.

Описание решения

Перебирать сами номера нельзя тк их очень много. Однако не нужен сам номер тк чтобы дописать следующую цифру, достаточно знать, на какой цифре мы сейчас стоим. Что было раньше, никак не влияет.

Поэтому в $dp[d]$ считаем сколько номеров текущей длины заканчивается на цифру d . В начале длина 1, и $dp[d] = 1$ для всех цифр, кроме 0 и 8 тк с них номер начинаться не может (по условию задачи).

Дальше на каждом шаге удлиняем номера на одну цифру - из каждой цифры d конь может уйти только в несколько соседних клеток, и все номера, кончавшиеся на d , переходят в номера, кончающиеся на них.

Асимптотика $O(N)$

Решение

```
M = 10 ** 9

moves = {
    0: [4, 6],
    1: [6, 8],
    2: [7, 9],
    3: [4, 8],
    4: [0, 3, 9],
    5: [],
    6: [0, 1, 7],
    7: [2, 6],
    8: [1, 3],
    9: [2, 4]
}

n = int(input())

dp = [0] * 10
for d in range(10):
    if d != 0 and d != 8:
        dp[d] = 1

for _ in range(n - 1):
    new_dp = [0] * 10
    for d in range(10):
        for x in moves[d]:
            new_dp[x] = (new_dp[x] + dp[d]) % M
    dp = new_dp

print(sum(dp) % M)
```

Описание проведенных тестов

Ввиду отсутствия в задании ссылки на платформу для проверки, тесты были сгенерированы через ЛЛМ (LLM были написаны тесты под задачу, а не под код - так называемое тестирование методом черного ящика).



```
dsa_labs — zsh — 112x13
ok 01_n1 0.01 c 7.9 M6
ok 02_n2 0.01 c 7.7 M6
ok 03_n3 0.01 c 7.7 M6
ok 04_n4 0.01 c 7.8 M6
ok 05_n10 0.01 c 7.5 M6
ok 06_n100 0.01 c 7.5 M6
ok 07_n999 0.01 c 7.5 M6
ok 08_n1000 0.01 c 7.7 M6
ok 09_n50 0.01 c 7.8 M6
OK 1.17 Ход конем lab1/1.17.py 9 тестов, макс 0.01 с (лимит 1 с), 8 МБ
Всего запусков: 9, провалов: 0
andrew@MacBook-Pro-Andrew dsa_labs %
```

Выводы

В рамках данной лабораторной работы эта задача показалась самой интересной. Было в очередной раз применено динамическое программирование для решения задачи.

Вывод

В рамках выполнения данной работы познакомились с жадными алгоритмами, узнали, что такое безопасный ход, научились с помощью него доказывать корректность жадного алгоритма. Так же поработали с динамическим программированием.